MOUNTAINS EXPEDITION MANAGEMENT SYSTEM

Relational Database Design Specification & Entity-Relationship (ER) Model

Target DBMS: PostgreSQL 16+ Relational Engine	**Core Entities:** 7 Normalized Relational Tables
Normalization Level: Third Normal Form (3NF)	**Analytical SQL Views:** 4 Live Aggregated Views
Integrity Constraints: 8 Foreign Keys with Cascade/Set Null	**Security:** PBKDF2 Password Hashing & Role RBAC

1. Executive DBMS Architecture Summary

The **MOUNTAINS** database architecture is engineered to provide high-concurrency expedition booking, comprehensive trekker safety tracking, partner gear hub integration, and 24/7 high-altitude rescue dispatch logistics. The relational schema is strictly normalized into **Third Normal Form (3NF)**, eliminating update, insertion, and deletion anomalies while enforcing ACID compliance for all financial and booking operations.

2. Conceptual Entities & Core System Relationships

The system centers around two primary entities: **USERS** (Trekkers, Certified Mountain Guides, and System Commanders) and **TREKS** (Expedition routes, difficulty ratings, capacity quotas, and base pricing). Transactional operations flow seamlessly through **BOOKINGS** (reservation agreements) and **TRANSACTIONS** (audit ledgers). Supporting operational workflows include **SHOPS** (partner rental vendors along trails), **RESCUE_OPERATIONS** (live GPS distress dispatches), and **REVIEWS** (verified trekker feedback).

3. Relational Structural Highlights

• **Primary Keys:** Every table utilizes a synthetic integer primary key (SERIAL) for performant join indexes.
• **Referential Integrity:** Foreign keys enforce operational reality—e.g. deleting a user is restricted if active bookings exist (ON DELETE RESTRICT), while deleting a trek cleans up associated itinerary bookings and vendor records (ON DELETE CASCADE).
• **Domain Constraints:** Enforced via PostgreSQL CHECK clauses ensuring valid difficulty tiers ('Easy', 'Moderate', 'Hard'), user roles ('admin', 'staff', 'user'), review star scores (1 to 5), and positive financial amounts.

4. Preview of Logical ER Diagram

Please turn to Page 2 for the full-spread, high-resolution landscape diagram with all attribute lists, key badges, and connector routes.

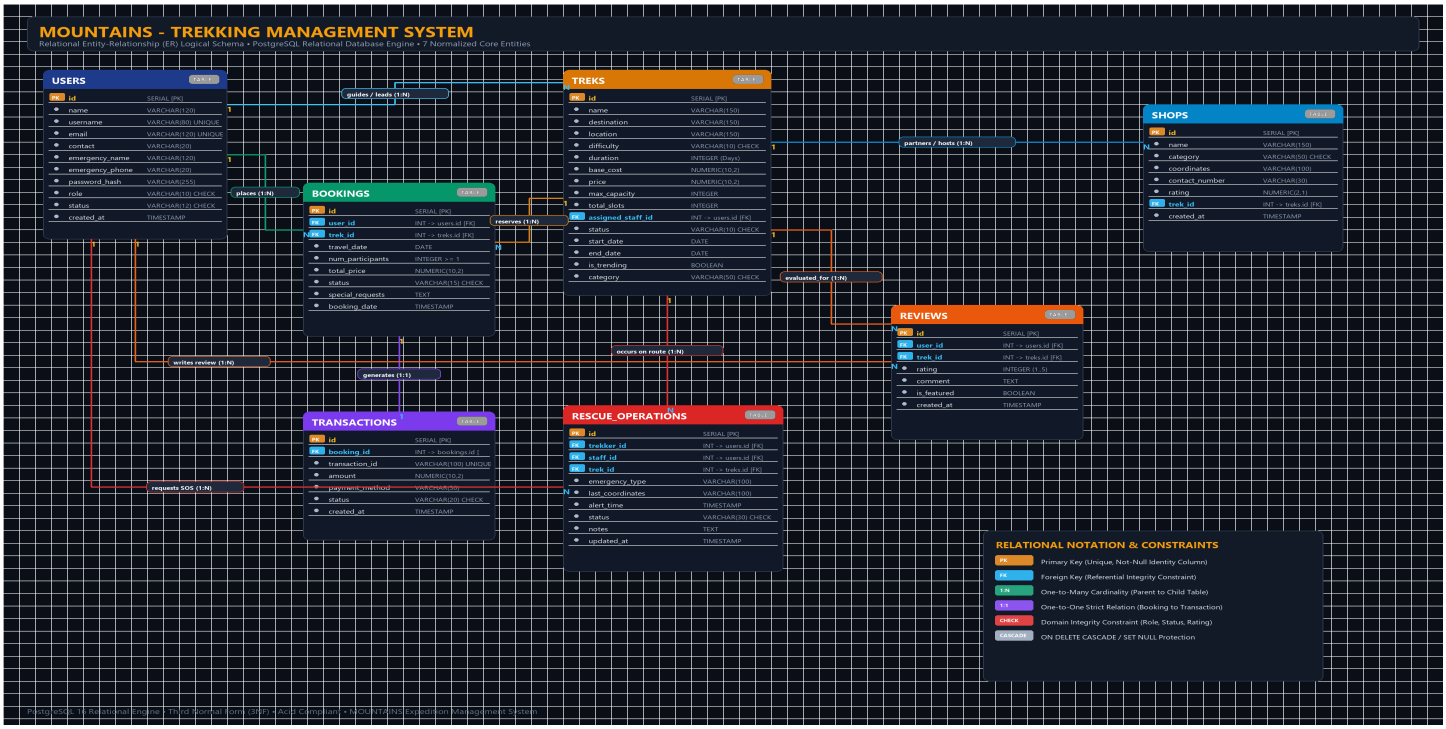
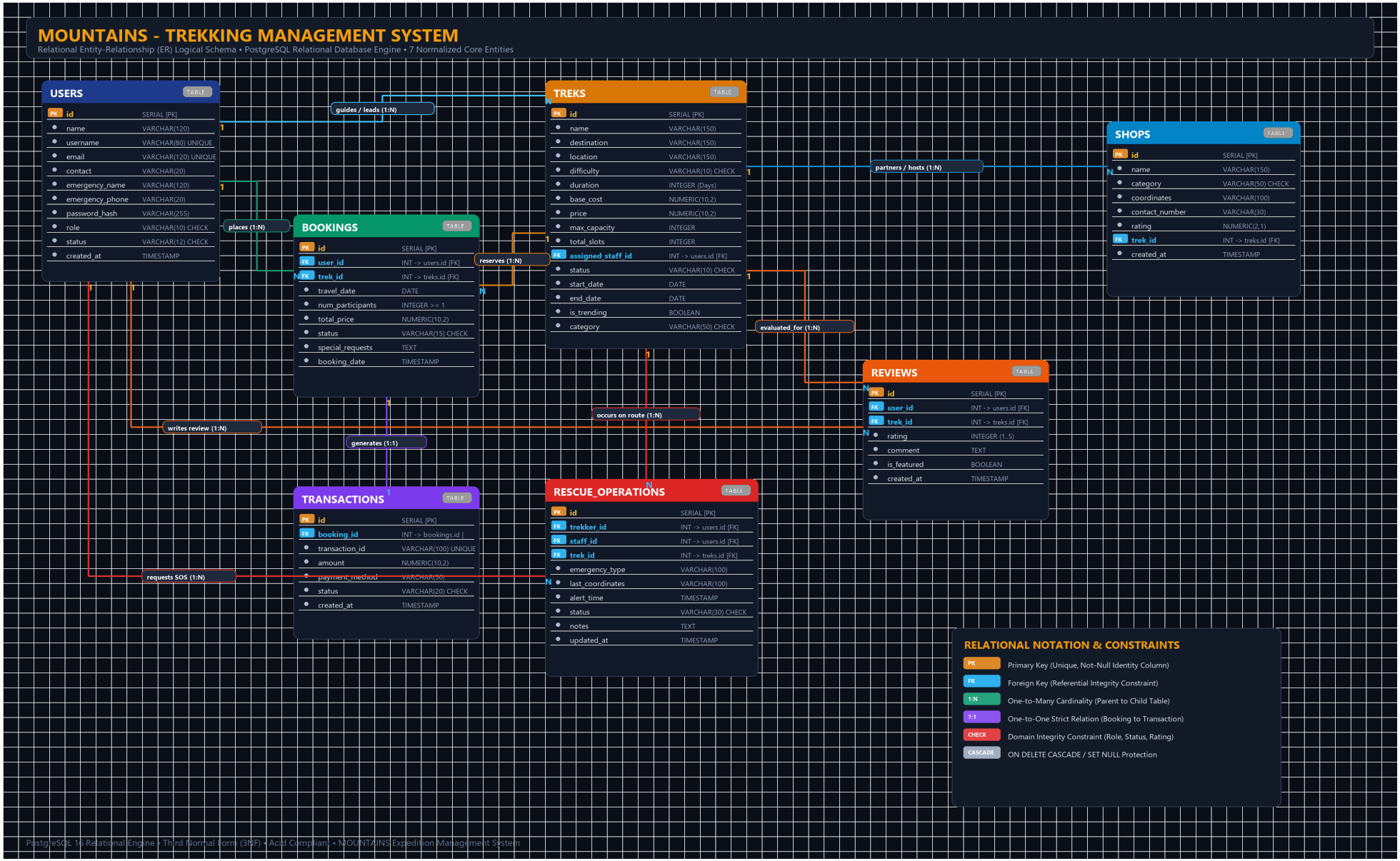


Figure 1: Full-Spread Relational Entity-Relationship (ER) Logical Schema

PostgreSQL Relational Engine • 7 Normalized Tables • Primary & Foreign Key Connectors • Cardinalities (1:N, 1:1)



5. Relational Data Dictionary & Attribute Specifications

Comprehensive catalog of tables, column data types, constraints, and operational business rules.

ENTITY 1: USERS — Trekkers, Mountain Guides & System Admins

Column	Type	Constraints & Keys	Description & Business Rules
id	SERIAL	**PRIMARY KEY**	Surrogate unique identity key for each user account.
name	VARCHAR(120)	**NOT NULL**	Full legal name of the user / staff guide / administrator.
username	VARCHAR(80)	**UNIQUE, NOT NULL**	Unique handle for system authentication and portal login.
email	VARCHAR(120)	**UNIQUE, NOT NULL**	User email address for booking invoices and SOS communications.
contact	VARCHAR(20)	**NULLABLE**	Direct mobile phone number with international country dialing code.
emergency_name	VARCHAR(120)	**NULLABLE**	Next of kin / emergency contact individual name.
emergency_phone	VARCHAR(20)	**NULLABLE**	High-priority phone number contacted during emergency rescues.
password_hash	VARCHAR(255)	**NOT NULL**	Secure PBKDF2 salted cryptographic password hash.
role	VARCHAR(10)	**CHECK (admin, staff, user)**	Role-Based Access Control: 'admin', 'staff', 'user'.
status	VARCHAR(12)	**CHECK (approved, pending, ...)**	Account lifecycle: staff requires admin approval before guiding.
created_at	TIMESTAMP	**NOT NULL, DEFAULT NOW()**	System audit timestamp indicating when account was registered.

ENTITY 2: TREKS — Alpine expeditions catalog with itineraries & guides

Column	Type	Constraints & Keys	Description & Business Rules
id	SERIAL	**PRIMARY KEY**	Unique expedition identifier.
name	VARCHAR(150)	**NOT NULL**	Marketing and official expedition name (e.g. Everest Base Camp).
destination	VARCHAR(150)	**NOT NULL**	Target mountain peak, valley, or country (e.g. Khumbu, Nepal).
location	VARCHAR(150)	**NOT NULL**	Geographical mountain region, state, or starting trailhead.
difficulty	VARCHAR(10)	**CHECK (Easy, Moderate, Hard)**	Physical difficulty tier based on altitude, incline, and terrain.
duration	INTEGER	**NOT NULL, CHECK (> 0)**	Expedition span in total calendar days.
base_cost	NUMERIC(10,2)	**NOT NULL, CHECK (>= 0)**	Base price per trekker for expedition permit and logistics.
max_capacity	INTEGER	**NOT NULL, CHECK (> 0)**	Maximum allowable group size for ecological preservation.
total_slots	INTEGER	**NOT NULL, CHECK (> 0)**	Configured capacity slots for the upcoming departure date.
assigned_staff_id	INTEGER	**FOREIGN KEY -> users.id**	Assigned Lead Alpine Guide (ON DELETE SET NULL).
status	VARCHAR(10)	**CHECK (Open, Closed, Completed)**	Expedition booking status.
start_date / end_date	DATE	**CHECK (end >= start)**	Expedition departure and conclusion dates.
is_trending	BOOLEAN	**DEFAULT FALSE**	Flag highlighting featured treks on landing page card grid.
category	VARCHAR(50)	**CHECK (Adventure, Trekking, ...)**	Trek classification for filtering in Search Tours.

ENTITY 3: BOOKINGS — Expedition reservation contracts linking Trekkers to Treks

Column	Type	Constraints & Keys	Description & Business Rules
id	SERIAL	**PRIMARY KEY**	Unique reservation booking number.
user_id	INTEGER	**FOREIGN KEY -> users.id**	Trekker placing the reservation (ON DELETE RESTRICT).
trek_id	INTEGER	**FOREIGN KEY -> treks.id**	Reserved expedition route (ON DELETE CASCADE).
travel_date	DATE	**NOT NULL, DEFAULT CURRENT_DATE**	Scheduled departure date for party arrival at basecamp.
num_participants	INTEGER	**NOT NULL, CHECK (> 0)**	Number of expedition slots reserved in party.
total_price	NUMERIC(10,2)	**NOT NULL, CHECK (>= 0)**	Computed price: num_participants multiplied by trek base_cost.
status	VARCHAR(15)	**CHECK (Confirmed, Booked, ...)**	Reservation status: 'Confirmed', 'Booked', 'Cancelled', 'Completed'.
special_requests	TEXT	**NULLABLE**	Special trekker dietary, high-altitude gear, or medical requests.

5. Relational Data Dictionary & Attribute Specifications (Continued)

Transactional and operational entities supporting financial ledgers, vendor hubs, emergency dispatches, and reviews.

ENTITY 4: TRANSACTIONS — Financial audit ledger and payment records

Column	Type	Constraints & Keys	Description & Business Rules
id	SERIAL	**PRIMARY KEY**	Unique transaction record identifier.
booking_id	INTEGER	**FOREIGN KEY -> bookings.id**	Associated booking reservation (ON DELETE CASCADE).
transaction_id	VARCHAR(100)	**UNIQUE, NOT NULL**	External bank reference code or gateway merchant tracking token.
amount	NUMERIC(10,2)	**NOT NULL, CHECK (>= 0)**	Processed currency amount.
payment_method	VARCHAR(50)	**NOT NULL**	Method: 'Credit Card', 'UPI/NetBanking', 'Apple Pay', 'PayPal'.
status	VARCHAR(20)	**CHECK (Success, Pending, ...)**	Financial status: 'Success', 'Pending', 'Refunded', 'Failed'.

ENTITY 5: SHOPS — Vetted gear rental, nutrition and photography basecamp hubs

Column	Type	Constraints & Keys	Description & Business Rules
id	SERIAL	**PRIMARY KEY**	Unique vendor shop identifier.
name	VARCHAR(150)	**NOT NULL**	Business name of partner shop / rental agency.
category	VARCHAR(50)	**CHECK (Gear Rental, Cafe, ...)**	Vendor service category (Gear Rental, Photography, Food, Medical).
coordinates	VARCHAR(100)	**NOT NULL**	GPS coordinates / basecamp trail marker for physically locating shop.
rating	NUMERIC(2,1)	**DEFAULT 4.8**	Aggregated customer feedback rating (scale 1.0 to 5.0).
trek_id	INTEGER	**FOREIGN KEY -> treks.id**	Expedition route near which shop operates (ON DELETE CASCADE).

ENTITY 6: RESCUE_OPERATIONS — 24/7 High-Altitude Emergency Logistics & SOS Dispatch

Column	Type	Constraints & Keys	Description & Business Rules
id	SERIAL	**PRIMARY KEY**	Unique emergency dispatch incident identifier.
trekker_id	INTEGER	**FOREIGN KEY -> users.id**	Distressed trekker needing emergency evacuation (ON DELETE CASCADE).
staff_id	INTEGER	**FOREIGN KEY -> users.id**	Assigned certified alpine guide/commander (ON DELETE SET NULL).
trek_id	INTEGER	**FOREIGN KEY -> treks.id**	Expedition route where distress occurred (ON DELETE SET NULL).
emergency_type	VARCHAR(100)	**NOT NULL**	Category: AMS Medical, Blizzard Stranded, Injury/Fall, Equipment.
last_coordinates	VARCHAR(100)	**NOT NULL**	Live GPS coordinates broadcast via Auto-GPS or landmark description.
status	VARCHAR(30)	**CHECK (Active, Dispatched, ...)**	Operation lifecycle: 'Active', 'Dispatched', 'Evacuated', 'Resolved'.

ENTITY 7: REVIEWS — Verified Trekker Experience Ratings & Testimonials

Column	Type	Constraints & Keys	Description & Business Rules
id	SERIAL	**PRIMARY KEY**	Unique review identifier.
user_id	INTEGER	**FOREIGN KEY -> users.id**	Verified trekker authoring the review (ON DELETE CASCADE).
trek_id	INTEGER	**FOREIGN KEY -> treks.id**	Expedition route evaluated (ON DELETE CASCADE).
rating	INTEGER	**CHECK (>= 1 AND <= 5)**	Numerical star rating from 1 (Poor) to 5 (Exceptional).
comment	TEXT	**NOT NULL**	Detailed text review detailing trail conditions, guides, and meals.

6. Relationship Cardinality & Referential Integrity Matrix

Referential integrity constraints prevent dangling foreign keys and dictate cascading lifecycle rules across entities.

Primary (Parent)	Foreign (Child)	Foreign Key Column	Type	On Delete	Relational Meaning
USERS	**TREKS**	assigned_staff_id	**1 : N**	SET NULL	One staff guide leads multiple expeditions.
USERS	**BOOKINGS**	user_id	**1 : N**	RESTRICT	User places bookings. User deletion restricted while bookings exist.
TREKS	**BOOKINGS**	trek_id	**1 : N**	CASCADE	An expedition receives multiple bookings from different trekkers.
BOOKINGS	**TRANSACTIONS**	booking_id	**1 : 1**	CASCADE	Each confirmed booking generates exactly one financial transaction record.
TREKS	**SHOPS**	trek_id	**1 : N**	CASCADE	An expedition route features multiple nearby gear rental shops.
USERS	**RESCUE_OPS**	trekker_id	**1 : N**	CASCADE	A trekker in distress broadcasts emergency SOS dispatch operations.
USERS	**RESCUE_OPS**	staff_id	**1 : N**	SET NULL	A staff guide is dispatched to coordinate rescue operations.
TREKS	**RESCUE_OPS**	trek_id	**1 : N**	SET NULL	A rescue occurs along a specific expedition trail or landmark.
USERS	**REVIEWS**	user_id	**1 : N**	CASCADE	Trekkers write verified reviews for completed expeditions.
TREKS	**REVIEWS**	trek_id	**1 : N**	CASCADE	Treks aggregate multiple 1 to 5 star ratings and feedback.

7. Relational SQL Views (Analytical Layer)

View Name	Underlying Source Tables	Business Purpose & SQL Joins
v_trek_availability	treks, bookings, users, reviews	Computes real-time booked participants via SUM(num_participants), live open slots (max_capacity - booked), guide name, and average star rating.
v_booking_details	bookings, users, treks, transactions	Denormalized reservation report joining trekker profile, emergency contacts, expedition destination, and gateway transaction IDs.
v_rescue_dashboard	rescue_operations, users, treks	Mission control dispatch log joining victim name, assigned guide name, mobile emergency contacts, and GPS distress coordinates.
v_trek_reviews	reviews, users, treks	Customer testimonials view linking review comments, star ratings, reviewer name, and associated expedition route.

8. Performance Indexing Strategy

To guarantee sub-millisecond query execution across large expedition datasets, B-Tree indexes are applied on all foreign key joins and frequent search filters:

• idx_bookings_trek & idx_bookings_user: Accelerate booking lookups by expedition or user.
• idx_treks_staff, idx_treks_trending, idx_treks_category: Fast landing page filtering.
• idx_rescue_trekker & idx_rescue_staff: Real-time mission control SOS dispatch queries.
• idx_shops_trek & idx_reviews_trek: Instant catalog joins on expedition detail views.